IHACK 2026

RECEIPT DON'T LIE

LAB SHEET

November 2026

Version 1.0

Prepared by

ECLOGIC SDN BHD

Table of Contents

Table of Contents.....1

1 Overview 2

2 Lab 1 : Read Before You Run 3

3 Lab 2 : Your First Terminal Commands 4

4 Lab 3: SSH Log Analysis and Brute-Force Detection 5

5 Lab 4 : Web Server Log Analysis.....5

6 Lab 5 : Mail Server Log Analysis 7

7 Lab 6: Cross-Log Correlation 8

1 Overview

This lab sheet provides practical exercises on network troubleshooting, system security, and performance tuning. Participants will learn essential techniques to diagnose connectivity issues, secure systems against threats, and optimize performance. Topics covered include hostname and IP configuration, network analysis tools, system hardening, and resource optimization. These labs are designed to enhance problem-solving skills and improve the security and efficiency of IT infrastructure.

2 Lab 1 : Read Before You Run

Objective:

This lab sheet provides hands-on exercises in introductory log analysis, command-line investigation, and multi-source incident correlation. Participants will learn essential techniques to inspect forensic log files, parse structured logs using core Linux utilities (grep, awk, sort, uniq), identify attack patterns across authentication, web, and mail services, and reconstruct end-to-end incident timelines.

Instructions:

1. The Golden Rule: Never run an unfamiliar command or script against a system before understanding every argument and flag. In incident response, log files serve as forensic evidence and must be inspected in a read-only manner.
2. Inspect the file permissions, ownership, and human-readable size:

```
ls -lh sample-auth.log
```

Expected finding: The file is approximately 33 KB with standard read access.

3. Verify the file format and encoding to ensure it is separate Bash label onto its own line or code block.

Expected output: sample-auth.log: ASCII text.

4. Determine the total line count of the file before processing it:

```
wc -l sample-auth.log
```

Expected output: Total record count (e.g., 300 sample-auth.log).

5. Preview the first 5 records to understand the log schema without flooding the terminal:

```
head -n 5 sample-auth.log
```

Expected output: The opening 5 syslog-formatted lines displaying the timestamp, host, process, and initial message strings.

3 Lab 2 : Your First Terminal Commands

Objective:

To learn fundamental text manipulation utilities (grep, awk, sort, uniq -c) and chain them together using pipes (|) to analyze structured log fields.

Instructions:

1. Review the core parsing command functions:
   a. grep: Searches input for lines matching a specified pattern.
   b. awk: Scans and processes columnar text based on field delimiters.
   c. sort: Sorts lines of text numerically or alphabetically.
   d. uniq -c: Filters duplicate adjacent lines and prefixes them with an occurrence count (requires sorted input).
   e. | (Pipe): Streams the standard output of one command directly into the standard input of the next.
2. Filter sample-auth.log for failed password events:

```
grep "Failed password" sample-auth.log
```

3. Extract the timestamp fields from the file using awk:

```
awk '{print $1, $2, $4, $5}' sample-auth.log | head -n 10
```

4. Chain multiple utilities together to isolate the targeted username field, aggregate occurrences, and output the results ranked by frequency

```
grep "Failed password" sample-auth.log | sed 's/.*for //' | sed 's/invalid user //' | awk '{print $1}' | sort | uniq -c | sort -nr
```

4 Lab 3: SSH Log Analysis and Brute-Force Detection

Objective:

To identify signs of automated SSH brute-force attacks in auth.log, isolate the primary suspect IP address, and confirm whether unauthorized access was achieved.

Instructions:

1. Count the total volume of failed authentication attempts within the log:

```
grep "Failed password" sample-auth.log | wc -l
```

Expected finding: A total of 164 failed entries across the systems, indicating brute-force or credential stuffing activity.

2. Extract and rank the source IP addresses generating the failed password attempts (field 4 from the end of the line, $(NF-3)):

```
grep "Failed password" sample-auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sort -nr | head -5
```

Expected finding: External IP 203.0.113.55 is ranked #1 with 12 targeted login attempts, clearly standing out from internal background noise.

3. Note key attack indicators: A single IP address attempting multiple administrative accounts (admin, root, backup, ubuntu) in rapid succession points to automated reconnaissance and credential guessing.
4. Check whether any login attempts succeeded from the suspect source:

```
grep "Accepted password" sample-auth.log
```

Expected finding: At 09:58:41, an accepted login session is recorded for user ubuntu originating from 203.0.113.55, confirming the server was compromised.

5 Lab 4 : Web Server Log Analysis

Objective:

Trace the suspect IP address (203.0.113.55) discovered in Lab 3 to investigate their activity on the web server

Instructions:

1. Filter the web access log down to the suspect IP to eliminate background noise:

```
grep "203.0.113.55" sample-access.log
```

Notice that the output reveals a small, distinct cluster of exactly 6 web requests made by this specific IP address between 10:05:03 and 10:06:02.

2. Extract the User-Agent field to see what software the attacker was using:

```
grep "203.0.113.55" sample-access.log | awk -F'"' '{print $6}' | sort | uniq -c
```

3. Command Breakdown:
    a. awk -F'"' tells the terminal to use quotation marks as the column delimiter.
    b. '{print $6}' extracts the 6th section, which holds the User-Agent string.
4. Expected finding:
    a. The result displays 6 requests with "Mozilla/5.0 (X11; Linux x86_64)". This indicates the attacker did not use an automated scanner (such as Nmap or sqlmap), but rather navigated the web server manually via a Linux web browser.
5. Extract the requested web pages (URIs) to determine the attacker's objective:

```
grep "203.0.113.55" sample-access.log | awk -F'"' '{print $2}'
```

6. What to look for (Chronological Activity):
    a. Initial Recon: GET / (Visited the homepage at 10:05:03)
    b. Directory Probe: GET /robots.txt (Returned 404 Not Found at 10:05:11)
    c. Admin Form Discovery: GET /wp-login.php (Accessed login portal at 10:05:18)
    d. Authentication Attempt: POST /wp-login.php (Submitted credentials at 10:05:26, resulting in a 302 redirect)

e. Unauthorized Access: GET /wp-admin/ and GET /wp-admin/profile.php (Successfully gained access to the WordPress administrative dashboard and profile area)

6 Lab 5 : Mail Server Log Analysis

Objective:

Identify automated abuse within the mail server logs and determine if the suspect IP attempted to compromise the email system.

Instructions:

1. Count the number of rejected delivery attempts:

When Postfix (the mail server) refuses to accept a message due to spam rules, blocklists, or relay restrictions, it logs a "NOQUEUE: reject" event.

```
grep "NOQUEUE: reject" sample-mail.log | wc -l
```

2. Expected finding:

The terminal will output 52, meaning the mail server blocked 52 unauthorized or invalid email attempts in this log sample.

3. Extract and rank the sender addresses to examine background noise:

```
grep "NOQUEUE: reject" sample-mail.log | awk -F"from=" '{print $2}' | awk '{print $1}' | sort | uniq -c | sort -nr | head -10
```

4. Command Breakdown:
   a. awk -F"from=" splits the log line at "from=".
   b. awk '{print $1}' isolates the specific email address enclosed in brackets.
   c. The output shows addresses like <admin@mail.test> and <backup@example.com> generating "550 5.1.1 User unknown" errors, representing ordinary baseline noise.
5. Tie it back to the suspect IP:

Search the mail log to check if the attacker IP (203.0.113.55) interacted with the mail server:

```
grep "203.0.113.55" sample-mail.log
```

5. What to look for:
   a. Connection: At 10:14:58, unknown[203.0.113.55] established an SMTP connection.
   b. Unauthorized Relay Attempt: At 10:15:01, the attacker attempted to send an email from=<attacker@evil.example> to=<victim@example.com>.
   c. Error Code: The server returned "554 5.7.1 Relay access denied".
   d. Outcome: The attacker attempted to abuse the server as an open mail relay for phishing/spam, but Postfix's security configuration successfully rejected the message before queuing.

7 Lab 6: Cross-Log Correlation

Objective: Using the authentication, web access, and mail logs, investigate the suspicious activity on 15 July 2026. Identify the source responsible for the activity, reconstruct the sequence of events across all three log sources, determine which authentication attempt succeeded, identify the subsequent web activity, and determine what occurred on the mail server. Support your conclusions with timestamps and log evidence.

Lab Needed:

- Terminal
- Logs:
  - sample-auth.log
  - sample-access.log
  - sample-mail.log

Instructions:

1. Download and copy the logs into your PC directory
2. Find repeated SSH failures
3. Start by looking for failed SSH authentication:

```
grep "Failed password" sample-auth.log
```

4. Count the source IPs producing failed logins:

```
grep "Failed password" sample-auth.log \
| grep -oE 'from ([0-9]{1,3}\.){3}[0-9]{1,3}' \
| awk '{print $2}' \
| sort \
| uniq -c \
```

```
| sort -nr
```

5. Which IP generates multiple failed SSH authentication attempts, and which usernames does it try?

6. Investigate the suspicious IP in authentication logs

```
grep "203.0.113.55" sample-auth.log
```

7. For chronological ordering:

```
grep "203.0.113.55" sample-auth.log | sort -k1,2
```

8. For chronological ordering:

```
grep "203.0.113.55" sample-auth.log | sort -k1,2
```

9. Look specifically for failures:

```
grep "203.0.113.55" sample-auth.log | grep "Failed password"
```

10. Then determine whether the source eventually authenticates successfully:

```
grep "203.0.113.55" sample-auth.log \
```

```
| grep -E "Failed password|Accepted"
```

11. Did the attack remain unsuccessful, or was there eventually a successful authentication?

```

```

12. Now use the discovered IP as the correlation key:

```
grep "203.0.113.55" \
  sample-auth.log \
  sample-access.log \
  sample-mail.log
```

13. Create a combined chronological view:

```
grep -h "203.0.113.55" \
  sample-auth.log \
  sample-access.log \
  sample-mail.log \
| sort -k1,2
```

14. Investigate web activity

```
grep "203.0.113.55" sample-access.log
```

15. Extract requests:

```
grep "203.0.113.55" sample-access.log \
| awk '{print $1, $2, $3, $6, $7, $9}'
```

16. Examine fields below:

```
timestamp
source IP
HTTP method
URI
HTTP status
```

17. Search for potentially interesting WordPress resources:

```
grep "203.0.113.55" sample-access.log \
| grep -E "wp-login|wp-admin"
```

18. Investigate mail activity

```
grep "203.0.113.55" sample-mail.log
```

19. Look for SMTP-related events:

```
grep "203.0.113.55" sample-mail.log \
| grep -E "connect|reject|relay"
```

20. Did the IP connect to the mail server? Was mail delivery/relay attempted? What was the result?

21. Build the complete attack timeline

```
for f in sample-auth.log sample-access.log sample-mail.log; do
    awk -v file="$f" '/203\.0\.113\.55/ {
        print $1, $2, "[" file "]", substr($0,index($0,$3))
    }' "$f"
done | sort -k1,2
```